

Week 6: RTOS Foundations - Detailed Notes

Table of contents

1	Introduction to Real-Time Operating Systems	2
1.1	What is an RTOS?	2
1.2	Bare-Metal vs RTOS	2
1.2.1	Bare-Metal (Super Loop) Architecture	2
1.2.2	RTOS Architecture	3
1.3	When to Use an RTOS	4
2	FreeRTOS Fundamentals	4
2.1	Task Concept	4
2.1.1	Task Control Block (TCB)	4
2.2	Task States	5
2.2.1	State Descriptions	5
2.3	Scheduler Operation	5
2.3.1	Priority-Based Preemptive Scheduling	5
2.3.2	Scheduling Example	6
2.3.3	Context Switching	6
2.4	Tick Timer	6
3	Task Creation and Management	7
3.1	Creating Tasks with CubeMX	7
3.2	Manual Task Creation	7
3.3	Task Function Structure	8
4	Timing and Delays	9
4.1	osDelay() - Relative Delay	9
4.2	vTaskDelayUntil() - Absolute Periodic Timing	9
4.3	HAL_Delay() - DO NOT USE IN TASKS	9
5	Priority Management	10
5.1	CMSIS-RTOS2 Priority Levels	10

5.2	Priority Guidelines	10
5.3	Dynamic Priority	10
6	Stack Management	11
6.1	Stack Size Selection	11
6.2	Stack Overflow Detection	11
6.3	Monitoring Stack Usage	12
7	Debugging FreeRTOS Applications	12
7.1	STM32CubeIDE RTOS Views	12
7.2	Common Issues and Solutions	12
7.3	Debug Hooks	12
8	Quick Reference	13
8.1	Essential API Functions	13
8.2	FreeRTOS Configuration Checklist	13
8.3	Task Template	14

1 Introduction to Real-Time Operating Systems

1.1 What is an RTOS?

A Real-Time Operating System (RTOS) is a specialized operating system designed for applications where timing is critical. Unlike general-purpose operating systems (Windows, Linux), an RTOS provides:

- **Deterministic timing:** Predictable worst-case response times
- **Task management:** Multiple independent threads of execution
- **Scheduling:** Automatic CPU time allocation based on priorities
- **Synchronization:** Safe resource sharing between tasks

1.2 Bare-Metal vs RTOS

1.2.1 Bare-Metal (Super Loop) Architecture

```
int main(void) {
    init_all_peripherals();

    while (1) {
        task_A(); // Takes X ms
    }
}
```

```

        task_B(); // Takes Y ms
        task_C(); // Takes Z ms
    }
}

```

Limitations:

- Response time = worst case loop time ($X + Y + Z$)
- Adding features increases loop time
- Time-critical operations require interrupts
- Complex interrupt-based designs are hard to debug

1.2.2 RTOS Architecture

```

void TaskA(void *arg) {
    while (1) {
        do_work_A();
        osDelay(period_A);
    }
}

void TaskB(void *arg) {
    while (1) {
        do_work_B();
        osDelay(period_B);
    }
}

int main(void) {
    init_all_peripherals();
    osThreadNew(TaskA, NULL, &attr_A);
    osThreadNew(TaskB, NULL, &attr_B);
    osKernelStart(); // Never returns
}

```

Advantages:

- Each task has independent timing
- High-priority tasks preempt lower ones
- Clean separation of concerns
- Easier to scale and maintain

1.3 When to Use an RTOS

Use RTOS When	Stay Bare-Metal When
Multiple independent timing requirements	Single simple function
Need responsive UI + background processing	Extreme timing constraints ($<10\mu\text{s}$)
Team development with module ownership	Very limited RAM ($<4\text{KB}$)
Complex state machines	Learning embedded basics
Scalability is important	Simple sensor reading + action

2 FreeRTOS Fundamentals

2.1 Task Concept

A **task** (also called a thread) is an independent unit of execution with:

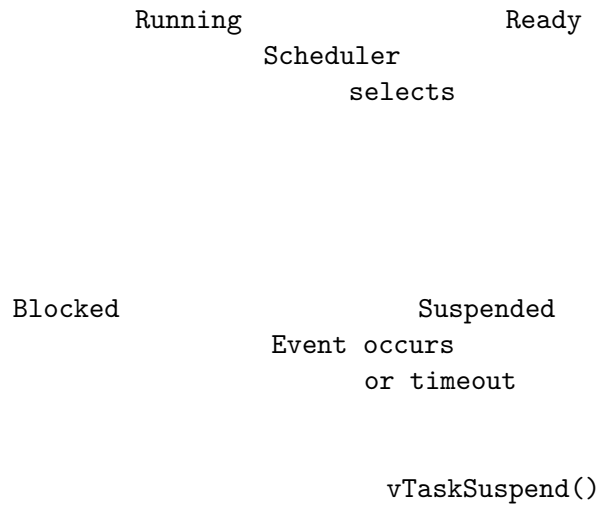
- **Entry function:** The code the task runs
- **Private stack:** Storage for local variables and call chain
- **Priority level:** Determines execution order
- **State:** Running, Ready, Blocked, or Suspended

2.1.1 Task Control Block (TCB)

FreeRTOS maintains a TCB for each task:

```
// Simplified TCB structure (internal to FreeRTOS)
typedef struct tskTaskControlBlock {
    volatile StackType_t *pxTopOfStack; // Current stack pointer
    ListItem_t xStateListItem;           // List membership
    ListItem_t xEventListItem;           // Event list membership
    UBaseType_t uxPriority;               // Task priority
    StackType_t *pxStack;                // Start of stack
    char pcTaskName[configMAX_TASK_NAME_LEN];
    // ... additional fields
} TCB_t;
```

2.2 Task States



2.2.1 State Descriptions

State	Description	Transitions From	Transitions To
Running	Currently executing	Ready (scheduler)	Ready, Blocked, Suspended
Ready	Waiting for CPU	Blocked, Running	Running
Blocked	Waiting for event/time	Running	Ready
Suspended	Manually paused	Any state	Ready (vTaskResume)

2.3 Scheduler Operation

2.3.1 Priority-Based Preemptive Scheduling

FreeRTOS uses a **preemptive priority-based scheduler**:

1. Highest priority **Ready** task always runs
2. If multiple tasks have same priority, they share time (round-robin)
3. Higher priority task becoming Ready **immediately preempts** lower priority

2.3.2 Scheduling Example

Time $\rightarrow$

Priority 3:

Priority 2:

Priority 1:

```

      ↑           ↑   ↑
P3 starts      P3 blocks P3 unblocks
                (osDelay) (delay done)

```

2.3.3 Context Switching

When switching between tasks, the scheduler:

1. Saves current task's CPU registers to its stack
2. Saves stack pointer to current task's TCB
3. Selects next task (highest priority Ready)
4. Loads stack pointer from new task's TCB
5. Restores CPU registers from new task's stack
6. Resumes execution at new task's last location

Context switch time on STM32F4 @ 84MHz: Approximately 1-5 μ s

2.4 Tick Timer

The RTOS scheduler runs from the **SysTick** timer interrupt:

- Default frequency: 1000 Hz (1ms tick)
- Each tick: scheduler checks for state changes
- Delays measured in ticks

```
// FreeRTOSConfig.h
#define configTICK_RATE_HZ    1000    // 1ms tick
#define configCPU_CLOCK_HZ    84000000
```

Important: Tick rate determines timing resolution. 1ms tick means:

- Minimum delay: 1ms
- Timing accuracy: $\pm 1\text{ms}$

3 Task Creation and Management

3.1 Creating Tasks with CubeMX

In STM32CubeMX, tasks are created in the FreeRTOS configuration:

1. Enable FreeRTOS (Middleware → FREERTOS → CMSIS_V2)
2. Add tasks in the Tasks tab
3. Configure: name, priority, stack size, entry function

Generated code appears in `freertos.c`:

```
/* Definitions for MyTask */
osThreadId_t MyTaskHandle;
const osThreadAttr_t MyTask_attributes = {
    .name = "MyTask",
    .stack_size = 128 * 4,
    .priority = (osPriority_t) osPriorityNormal,
};

void MX_FREERTOS_Init(void) {
    MyTaskHandle = osThreadNew(StartMyTask, NULL, &MyTask_attributes);
}

void StartMyTask(void *argument) {
    for(;;) {
        osDelay(1);
    }
}
```

3.2 Manual Task Creation

Without CubeMX, create tasks manually:

```
// Using FreeRTOS native API
TaskHandle_t taskHandle;
BaseType_t result = xTaskCreate(
    TaskFunction,          // Function pointer
    "TaskName",            // Name (for debugging)
    128,                   // Stack size in words
    NULL,                  // Parameter
    tskIDLE_PRIORITY + 2, // Priority (0 = idle)
```

```

    &taskHandle          // Handle storage
);

// Using CMSIS-RTOS2 API (recommended for STM32)
osThreadId_t taskHandle;
const osThreadAttr_t taskAttr = {
    .name = "TaskName",
    .stack_size = 128 * 4, // In bytes
    .priority = osPriorityNormal
};
taskHandle = osThreadNew(TaskFunction, NULL, &taskAttr);

```

3.3 Task Function Structure

```

void MyTask(void *argument)
{
    // === INITIALIZATION PHASE ===
    // Runs once when task starts
    uint32_t counter = 0;

    // === MAIN LOOP ===
    for(;;) // Infinite loop (normal for RTOS tasks)
    {
        // Task work
        counter++;
        do_something();

        // CRITICAL: Must block or delay!
        osDelay(100); // Release CPU to other tasks
    }

    // === CLEANUP (rarely reached) ===
    // If task ever exits loop:
    osThreadTerminate(NULL);
}

```

4 Timing and Delays

4.1 osDelay() - Relative Delay

```
osDelay(500); // Block for 500 ticks (500ms at 1kHz)
```

Behavior:

- Task enters Blocked state
- CPU available for other tasks
- Task returns to Ready after delay

Timing accuracy: Within 1 tick of specified delay

4.2 vTaskDelayUntil() - Absolute Periodic Timing

```
TickType_t lastWakeTime = xTaskGetTickCount();
for(;;) {
    // Task work here (takes variable time)
    vTaskDelayUntil(&lastWakeTime, pdMS_TO_TICKS(100));
}
```

Use case: Precise periodic execution (e.g., control loops)

Difference from osDelay:

osDelay	vTaskDelayUntil
Delays FROM now	Delays UNTIL next period
Period = work time + delay	Period = constant
Timing drifts	No drift

4.3 HAL_Delay() - DO NOT USE IN TASKS

```
// WRONG!
void BadTask(void *arg) {
    for(;;) {
        HAL_GPIO_Toggle(...);
    }
}
```

```

    HAL_Delay(100); // Busy-wait! CPU is 100% busy!
}
}

```

Problem: HAL_Delay() uses a busy-wait loop. The CPU cannot run other tasks during the delay.

Solution: Always use osDelay() in RTOS tasks.

5 Priority Management

5.1 CMSIS-RTOS2 Priority Levels

Name	Value	Typical Use
osPriorityIdle	1	Idle task only
osPriorityLow	8	Background tasks
osPriorityBelowNormal	16	Low importance
osPriorityNormal	24	Most tasks
osPriorityAboveNormal	32	Slightly elevated
osPriorityHigh	40	Time-sensitive
osPriorityRealtime	48	Hard real-time

5.2 Priority Guidelines

1. **Start with Normal:** Most tasks should be osPriorityNormal
2. **Raise only when needed:** Only increase priority if response time is critical
3. **Avoid priority inversion:** Be careful with shared resources (covered in Week 7)
4. **Document priority decisions:** Comment why each task has its priority

5.3 Dynamic Priority

Change priority at runtime:

```

// Get current priority
osPriority_t current = osThreadGetPriority(taskHandle);

// Set new priority
osThreadSetPriority(taskHandle, osPriorityHigh);

```

6 Stack Management

6.1 Stack Size Selection

Each task needs stack space for:

Item	Typical Words
Local variables	Varies
Function call frames	8-16 per call depth
Context save (registers)	16-32
Interrupt overhead	0-32

Rule of thumb:

- Simple tasks: 128 words (512 bytes)
- Medium tasks: 256 words (1024 bytes)
- Complex tasks (printf, deep calls): 512+ words

6.2 Stack Overflow Detection

Enable in FreeRTOSConfig.h:

```
#define configCHECK_FOR_STACK_OVERFLOW 2
```

Implement handler:

```
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
{
    // Task name available for debugging
    (void)pcTaskName;

    // Options:
    __BKPT(0);           // Halt debugger
    NVIC_SystemReset();  // Reset MCU
    while(1);            // Hang (with watchdog will reset)
}
```

6.3 Monitoring Stack Usage

Use high water mark to check actual usage:

```
UBaseType_t stackRemaining = uxTaskGetStackHighWaterMark(taskHandle);  
// Returns minimum free stack ever recorded  
// If this is low, increase stack size!
```

7 Debugging FreeRTOS Applications

7.1 STM32CubeIDE RTOS Views

Enable in debug configuration:

1. Window → Show View → FreeRTOS → Task List
2. Window → Show View → FreeRTOS → Queue List
3. Window → Show View → FreeRTOS → Semaphore List

Task List shows:

- Task name
- Current state
- Priority
- Stack high water mark
- Run time (if enabled)

7.2 Common Issues and Solutions

Problem	Symptom	Solution
HAL_Delay in task	System unresponsive	Use osDelay()
No osDelay/block	Other tasks never run	Add blocking call
Stack overflow	Random crashes	Increase stack, enable detection
Priority inversion	High-priority task blocked	Use mutex (Week 6)
Race condition	Intermittent data corruption	Use mutex or queue (Week 6)

7.3 Debug Hooks

Enable in CubeMX for runtime monitoring:

```
// Called when entering idle (CPU has nothing to do)
void vApplicationIdleHook(void)
{
    // Can toggle LED to show CPU is free
    // Can enter low-power mode
}

// Called on malloc failure
void vApplicationMallocFailedHook(void)
{
    // Heap exhausted!
    __BKPT(0);
}
```

8 Quick Reference

8.1 Essential API Functions

Function	Purpose
osThreadNew()	Create a task
osDelay()	Delay task (relative)
osKernelStart()	Start scheduler
osThreadGetId()	Get current task handle
osThreadSetPriority()	Change task priority
osThreadTerminate()	Delete a task

8.2 FreeRTOS Configuration Checklist

Setting	Recommended	Notes
configTICK_RATE_HZ	1000	1ms resolution
configMAX_PRIORITIES	7	Usually sufficient
configMINIMAL_STACK_SIZE	128	Words, not bytes
configTOTAL_HEAP_SIZE	15360	Adjust based on needs
configUSE_PREEMPTION	1	Required for real-time
configCHECK_FOR_STACK_OVERFLOW	2	Enable for debugging

8.3 Task Template

```
void MyTask(void *argument)
{
    // Initialization

    for(;;)
    {
        // Task work

        osDelay(PERIOD_MS); // Or block on event
    }
}
```